DATASTRUCTURE

QUEUES:-

SIMPLE QUEUE

```
#include <iostream>
using namespace std;

class Queue
{
   int A[10], front, rear;

public:
   Queue()
   {
      front = -1;
      rear = -1;
   }

   void enqueue(int x)
   {
      if (rear == 9)
```

```
        cout << "Queue Overflow";
    else
    {
        if (front == -1)
            front = 0;

        rear++;
        A[rear] = x;
    }
}

void dequeue()
{
    if (front == -1 || front > rear)
        cout << "Queue Underflow";
    else
    {
        cout << "Deleted Element: " << A[front];
        front++;
    }
}
```

```
    void display()
    {
        if (front == -1 || front > rear)
            cout << "Queue is Empty";
        else
        {
            for (int i = front; i <= rear; i++)
                cout << A[i] << " ";
        }
    }
};

int main()
{
    Queue q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
```

```
    cout << "Queue Elements: ";
    q.display();

    cout << endl;

    q.dequeue();

    cout << endl << "After Deletion: ";
    q.display();

    return 0;
}
```

Output:

```
Queue Elements: 10 20 30
Deleted Element: 10
After Deletion: 20 30
```

DOUBLE ENDED QUEUE

```
#include <iostream>
using namespace std;

class Deque
{
```

```
    int A[10], front, rear;

public:
    Deque()
    {
        front = -1;
        rear = -1;
    }

    void insertFront(int x)
    {
        if (front == 0)
            cout << "Overflow";
        else
        {
            if (front == -1)
                front = rear = 0;
            else
                front--;

            A[front] = x;
```

```
    }
}

void insertRear(int x)
{
    if (rear == 9)
        cout << "Overflow";
    else
    {
        if (front == -1)
            front = rear = 0;
        else
            rear++;

        A[rear] = x;
    }
}

void deleteFront()
{
    if (front == -1)
```

```
        cout << "Underflow";
    else if (front == rear)
        front = rear = -1;
    else
        front++;
}

void deleteRear()
{
    if (rear == -1)
        cout << "Underflow";
    else if (front == rear)
        front = rear = -1;
    else
        rear--;
}

void display()
{
    if (front == -1)
        cout << "Deque is Empty";
```

```
        else
        {
            for (int i = front; i <= rear; i++)
                cout << A[i] << " ";
        }
    }
};

int main()
{
    Deque d;

    d.insertRear(10);
    d.insertRear(20);
    d.insertFront(5);

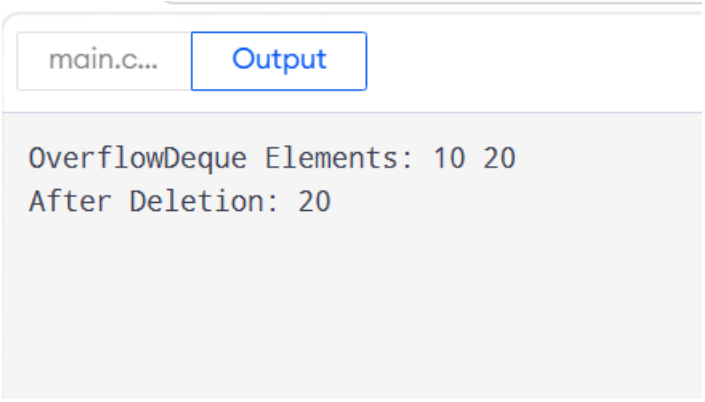
    cout << "Deque Elements: ";
    d.display();

    d.deleteFront();
```

```
    cout << "\nAfter Deletion: ";
    d.display();

    return 0;
}
```

Output:



CIRCULAR QUEUE

```
#include <iostream>
using namespace std;

class CircularQueue
{
    int A[10], front, rear;

public:
    CircularQueue()
    {
```

```
    front = -1;
    rear = -1;
}

void enqueue(int x)
{
    if ((rear + 1) % 10 == front)
        cout << "Queue Overflow";
    else
    {
        if (front == -1)
            front = 0;

        rear = (rear + 1) % 10;
        A[rear] = x;
    }
}

void dequeue()
{
    if (front == -1)
```

```
        cout << "Queue Underflow";
    else
    {
        cout << "Deleted Element: " << A[front];

        if (front == rear)
            front = rear = -1;
        else
            front = (front + 1) % 10;
    }
}

void display()
{
    if (front == -1)
        cout << "Queue is Empty";
    else
    {
        int i = front;

        while (true)
```

```
        {
            cout << A[i] << " ";

            if (i == rear)
                break;

            i = (i + 1) % 10;
        }
    }
  }
};

int main()
{
  CircularQueue q;

  q.enqueue(10);
  q.enqueue(20);
  q.enqueue(30);

  cout << "Circular Queue: ";
```

```
    q.display();

    cout << endl;

    q.dequeue();

    cout << endl << "After Deletion: ";
    q.display();

    return 0;
}
```

Output:

